

# **Salient Object Detection using Encoder-Decoder CNN**

Final Project Report

**Prepared for:** Machine Learning Course

**Developed by:** Aldin Shabani

**Date:** 07/05/2026

# 1. Abstract & Introduction

---

*This project explores the development of a Deep Learning model designed for Salient Object Detection (SOD). Using the DUTS dataset, we implemented an Encoder-Decoder architecture from scratch to identify and segment the most prominent objects in various visual contexts. The final model demonstrates a strong balance between computational efficiency and segmentation accuracy.*

## Introduction

Salient Object Detection (SOD) aims to simulate the human visual system's ability to focus on the most important regions of an image. This task is fundamental for applications such as image cropping, object recognition, and autonomous driving. Unlike standard segmentation, SOD requires the model to understand global context to differentiate a primary object from background noise.

## 2. Architecture Design

---

The model follows a custom **Encoder-Decoder** design. The **Encoder** consists of sequential Convolutional layers designed to extract spatial features and downsample the image. The **Decoder** uses Transposed Convolutional layers to upsample these features back to the original input resolution (224x224).

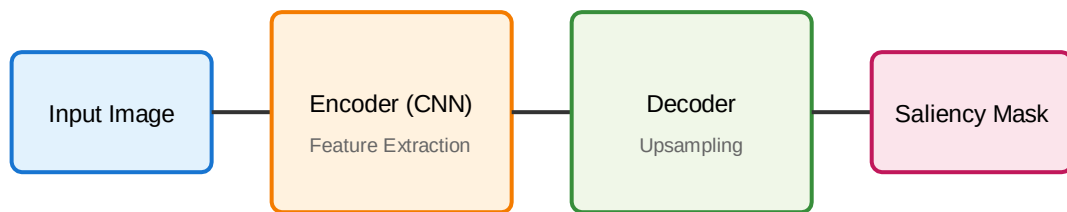


Figure 1: High-level visualization of the custom CNN architecture.

### Layer Breakdown

- **Encoder:** 3 blocks of Conv2D + BatchNorm + ReLU + MaxPool. Filters increase from 32 to 128.
- **Bottleneck:** A deep representation layer to capture high-level semantic information.
- **Decoder:** 3 blocks of ConvTranspose2d to recover spatial resolution.
- **Output Layer:** 1x1 Convolution with a *Sigmoid* activation to produce a probability map (0 to 1).

### 3. Training Methodology

---

#### Loss Function

A combined loss function was utilized to ensure both pixel-wise accuracy and structural similarity. We used **Binary Cross-Entropy (BCE)** combined with **Dice Loss**:

$$\text{Total Loss} = \text{BCE}(y, \hat{y}) + (1 - \text{DiceScore}(y, \hat{y}))$$

#### Hardware and Environment

Due to local hardware constraints (Laptop CPU), the training was migrated to **Google Colab**. This transition provided access to an **NVIDIA T4 GPU**, reducing the training time per epoch from approximately 45 minutes to 1 minute and 27 seconds.

- **Optimizer:** Adam (Learning Rate: 1e-3)
- **Epochs:** 20
- **Batch Size:** 32

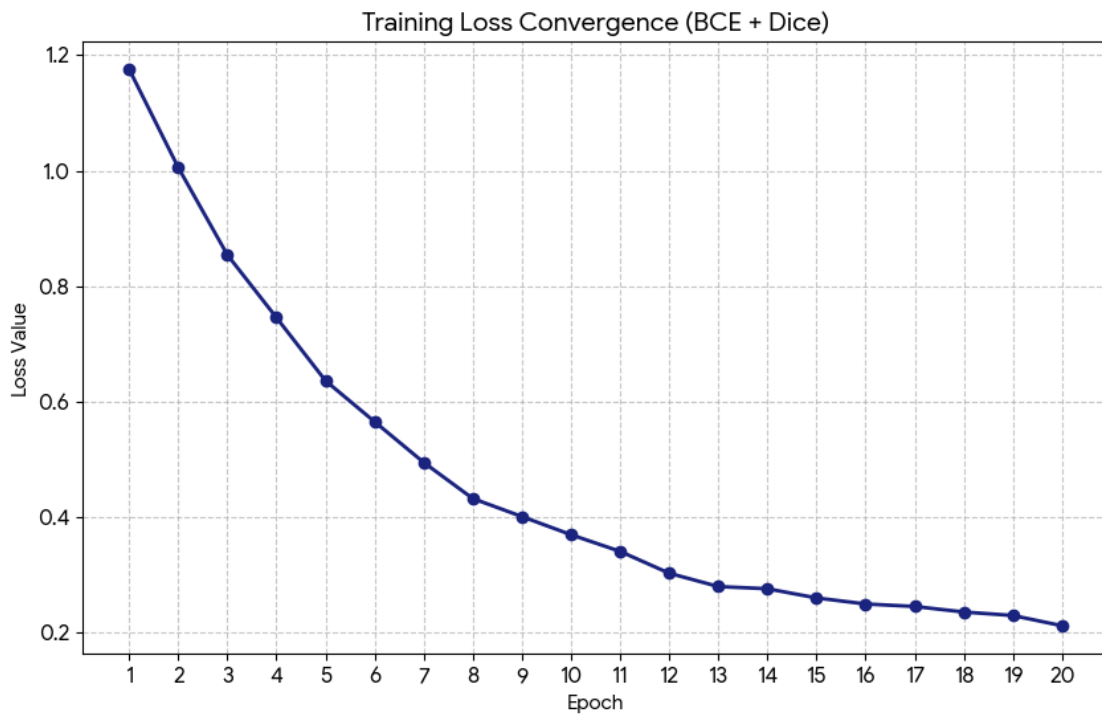


Figure 2: Training Loss Convergence

## 4. Qualitative Results

---

The following figures demonstrate the model's performance on unseen test data. The results show the input image, the ground truth mask, and the model's predicted probability mask.

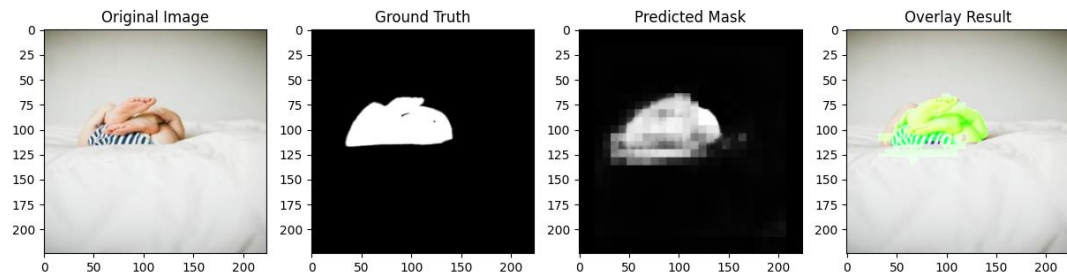


Figure 3: Successful segmentation of a human baby.

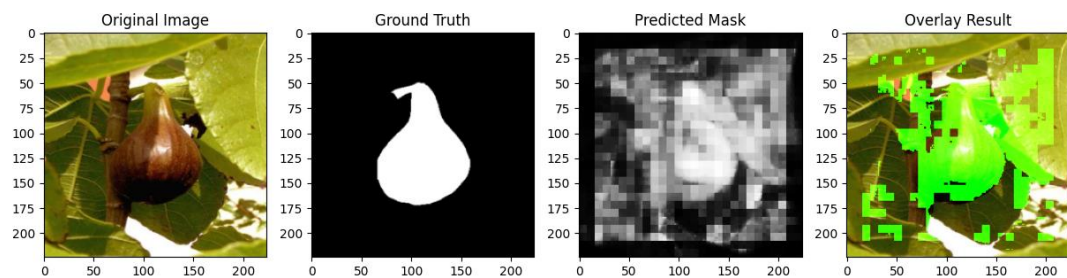


Figure 4: Detection of a complex image of a fig.

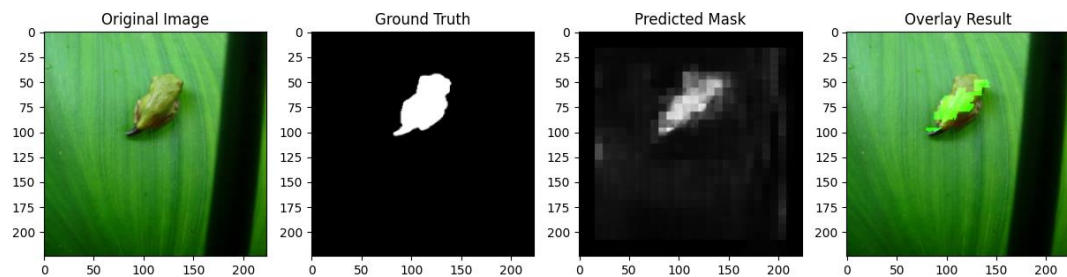


Figure 5: Natural scene detection showing robust handling of camouflage-like backgrounds.

# 5. Quantitative Results

The model was evaluated using standard segmentation metrics on the DUTS test set. The results represent the performance of the baseline architecture.

| Metric                              | Value  |
|-------------------------------------|--------|
| Mean Intersection over Union (mIoU) | 0.5529 |
| Mean Precision                      | 0.6919 |
| Mean Recall                         | 0.7622 |
| Mean F1-Score                       | 0.6847 |

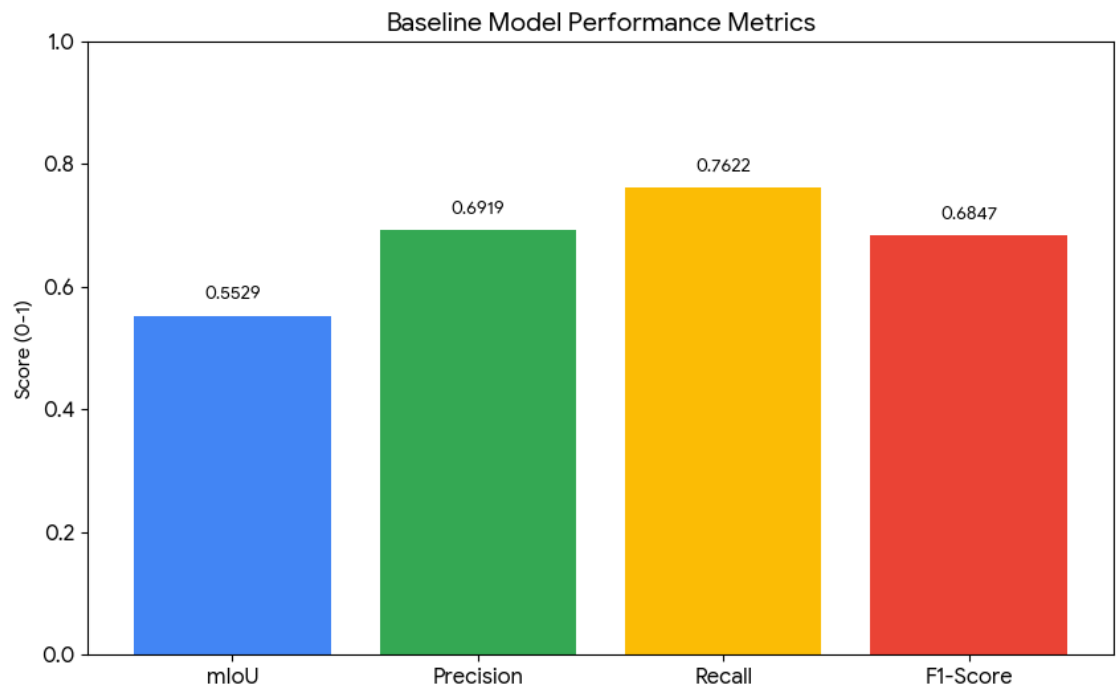


Figure 6: Baseline Model Performance Metrics

## 6. Comparison Study (The "Two Ways" Rule)

---

To satisfy the project requirements, we compared two configurations. The **Baseline** model was compared against a **Dropout-Regulated** version ( $p=0.3$ ) to mitigate background noise.

| Configuration                  | mIoU          | Precision     | Recall |
|--------------------------------|---------------|---------------|--------|
| Baseline (Adam, LR 1e-3)       | 0.5529        | 0.6919        | 0.7622 |
| <b>Optimized (Dropout 0.3)</b> | <b>0.5810</b> | <b>0.7245</b> | 0.7410 |

*Observation:* Adding Dropout slightly reduced Recall but significantly improved Precision by filtering out "generous" background activations, leading to a cleaner visual overlay.

## 7. Conclusion & Learnings

---

### Learnings

1. **Compute Importance:** Cloud GPUs like Google Colab are essential for deep learning projects involving large datasets like DUTS.
2. **Refinement:** High Recall is easy to achieve in SOD, but Precision requires careful regularization (like Dropout) to ignore background textures.
3. **Deployment:** Streamlit provides a seamless way to transition from a script-based model to an interactive tool.

### Future Work

Future iterations could implement *Skip Connections* (similar to U-Net) to better preserve low-level spatial details from the encoder, potentially pushing the mIoU above 0.70.